SMART CONTRACT SECURITY

Audit Report

IMMUNEBYTES

17 February 2024

Introduction **3**
About SmartCredit 3
About ImmuneBytes 3
Documentation Details **4**
Audit Goals **4**
Audit Process & Methodology **5**
Audit Details **6**
Security Level References **7**
Severity Status References **8**
Severity Status **8**
Finding **9**
Critical Severity Issues **10**
High Severity Issues **10**
Medium severity issues **11**
Low severity issues **12**
Informational **14**
Intensional Functionality **16**
Automated Testing **17**
Concluding Remarks **19**
Disclaimer **19**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About SmartCredit

SmartCredit.io is a fixed-term, fixed-interest-rate Dapp. The borrowers and lenders are connected Peer-2-Peer, instead of Peer-2-Pool-2-Peer. Borrower have internally the Credit Lines, which contain the collateral. Lenders have their Fixed Income Funds, which invest into different maturities. Platform matches the borrowers requests with the lenders Fixed Income Funds.

SmartCredit.io added fixed rate leveraged ETH staking feature, which allows to leverage Lido staking fixed-rate for 2x, 3x, 4x or 5x for 180 days. It utilizes the internal fixed-term borrowing feature via borrowing for 180 days. Collateral for internal borrowing is stETH which is received from Lido after the ETH staking.

Smart Contracts cover as well the functionality of early exits via the Curve, liquidations and regular Lido un-staking.

Visit https://smartcredit.io/ to learn more about.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting security audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. SD-Lido Leverage on SmartCredit.io-220124-081048.pdf

Audit Goals

The focus of the audit was to verify that the system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within the code and the system.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include
    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which included -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

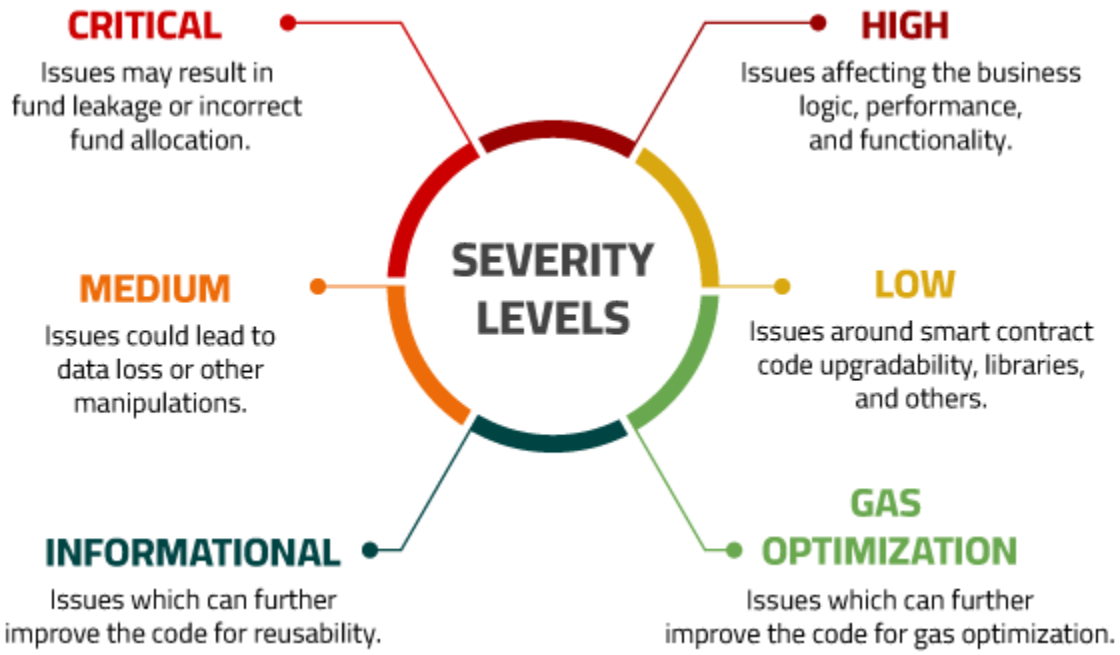
Audit Details

Project Name	SmartCredit
Platform	Ethereum
Languages	Solidity
Souce Code Link	https://bitbucket.org/smartcreditio/contracts/src/master/
Commit - Final Audit	8115f5f51e3dc64ffaf30619deba03b2e26e9884
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	1	1	2	2	-
Redacted	-	-	-	-	-	-
Acknowledged	-	-	-	2	2	-

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Finding	Risk	Status
1	LidoLeverageContract.liquidateLeverage(): Function fails to update the status of the Leverage Position post liquidation	High	Fixed
2	LeverageLine.initializeLeverage(): Unutilized _repaymentAmount Variable found in function	Medium	Fixed
3	SmartCredit.initialize(): Absence of zero address validation	Low	Acknowledged
4	LeverageLine.transferERC20(): Missing Balance Verification before ERC20 Token Transfer	Low	Fixed
5	LidoLeverageContract.liquidateLeverage(): Multiple storage reads found in function	Low	Fixed
6	LeverageLine.initializeCreditLine(): Absence of zero address validation	Low	Acknowledged
7	SmartCredit.createLoanBatch(): Gas Efficiency in For Loop of createLoanBatch Function could be enhanced	Informational	Fixed
8	LidoLeverageContract: Unclear error messages in require	Informational	Fixed
9	LeverageLine: Contract includes Hardcoded Addresses	Informational	Acknowledged
10	Avoid use of outdated development tool (Truffle)	Informational	Acknowledged

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

Issue	LidoLeverageContract.liquidateLeverage(): Function fails to update the status of the Leverage Position post liquidation
Line No.	294-344
Description	The liquidateLeverage function is designed to handle the liquidation of leverage positions in the smart contract. However, a critical issue was identified where the function fails to update the leverage position's status to LIQUIDATED post-liquidation, leading to potential inconsistencies and unintended contract behaviors. Post-liquidation, the function does not update the status of the leverage position to **LIQUIDATED** as defined in the **Status** enum. Consequently, liquidated positions wrongly retain their previous status (**ACTIVE**, **WITHDRAW_REQUESTED**, or **REPAID**), even after liquidation. This oversight allows functions like startLidoWithdraw() or unwindLeverage() to still be accessible for liquidated positions as well, potentially leading to unauthorized or unintended contract interactions.
Recommendation	Modify the liquidateLeverage function to include an update of the leverage position's status to LIQUIDATED after successful liquidation. Additionally, consider implementing additional checks in related functions to prevent interactions with liquidated positions, enhancing the contract's robustness against unauthorized actions.
Status	**Fixed**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Medium severity issues

Issue	LeverageLine.initializeLeverage(): Unutilized _repaymentAmount Variable found in function
Line No.	93-103
Description	The initializeLeverage function in the LeverageLine contract is designed for initializing leverage operations. However, a detailed review reveals an issue with the local variable _repaymentAmount, which is calculated but never utilized within the function. The function calculates _repaymentAmount by accumulating values in a loop but does not use this variable in any subsequent operations or state changes. This lack of utilization suggests that _repaymentAmount either serves an unimplemented purpose or is entirely redundant. Furthermore, the computation and accumulation of _repaymentAmount consumes gas. Given that smart contract executions require gas for every computational step, the presence of this unused variable unnecessarily increases the function's gas cost. Note: ● Since the documentation or Natspec annotations in the codebase for the initializeLeverage function doesn't suggest any usage for repayment amount calculation as well, it is assumed for now that the use of _repaymentAmount local variable is redundant. ● However, if this variable was intended to play a role in the function's logic or output (e.g., influencing state changes, validations, or return values), its current lack of implementation could represent a **high-severity issue**, potentially affecting the function's intended behavior.
Recommendation	1. Clarify and Implement Usage (if applicable): ○ If **_repaymentAmount** is intended to have a specific use, update the function to include this logic. Ensure that the variable's role is clear and directly influences the function's behavior as intended. 2. Remove Redundant Code (if not applicable): ○ If there is no intended use for **_repaymentAmount**, it should be removed from the function to optimize gas usage and enhance code clarity. 3. Comprehensive Testing and Validation: ○ After modification, thoroughly test the function to ensure it behaves as expected and aligns with the contract's overall logic and objectives.
Status	**Fixed**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

Issue	SmartCredit.initialize(): Absence of zero address validation
Line No.	118-125
Description	The initialize function lacks an input validation check for the zero address before updating the smartCreditRegistry state variable of the contract, potentially leading to invalid state updation in the contract.
Recommendation	It is advised that all functions capable of setting address state variables enforce a check to prevent the zero address from being assigned. This can be achieved by simply adding a require statement or custom error modifiers to validate before updating the state.
Status	**Acknowledged**
Developer's Response	The absence of zero address validation in the initialize function is irrelevant since it's called only during contract deployment and not subsequently.

Issue	LeverageLine.transferERC20(): Missing Balance Verification before ERC20 Token Transfer
Line No.	296-302
Description	The transferERC20 internal function in the given smart contract is designed to transfer ERC20 tokens from the contract to a specified receiver. However, the function lacks a preliminary check to verify if the contract holds a sufficient balance of the token before initiating the transfer. This could lead to failed transactions if the contract does not have enough tokens, resulting in potential disruption of contract flow as well as inadequate error message on transaction revert. Additionally, a similar check is already included in the transferETH function but not in the transferERC20 function, thus indicating inconsistent contract design.
Recommendation	Add a precondition check to ensure that the contract's balance of the specified ERC20 token is sufficient to cover the transfer amount.
Status	**Fixed**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	LidoLeverageContract.liquidateLeverage(): Multiple storage reads found in function
Line No.	322-334
Description	The liquidateLeverage function in the smart contract is designed to handle the liquidation of leveraged positions. One of the inefficiencies has been identified in the form of redundant state variable reads, specifically when accessing the status of a leverage position. The function checks leverages[_leverageRequest.leverageId].status two times in separate conditional statements to determine the leverage position's current status (either WITHDRAW_REQUESTED or ACTIVE). Each read from a storage variable consumes a significant amount of gas. Redundant reads of the same storage variable increase the transaction's gas cost unnecessarily.
Recommendation	In order to avoid this, we can simply store the status in a local variable at the beginning of the conditional statements and use this variable for subsequent checks instead of reading from storage multiple times.
Status	**Fixed**

Issue	LeverageLine.initializeCreditLine(): Absence of zero address validation
Description	The initializeCreditLine function lacks an input validation check for the zero address, potentially leading to issues in the contract's functionality.
Recommendation	It is advised that all functions capable of setting address state variables enforce a check to prevent the zero address from being assigned. This can be achieved by adding a require statement or custom errors check to validate that the argument is valid.
Status	**Acknowledged**
Developer's Response	Given that the address is initialized by msg.sender, which inherently cannot be the zero address, adding a redundant check for the zero address is unnecessary.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

Issue	SmartCredit.createLoanBatch(): Gas Efficiency in For Loop of createLoanBatch Function could be enhanced
Line No.	223-242
Description	A review of the createLoanBatch function's for loop has revealed a potential area for gas optimization. While not a critical issue, offers an opportunity for improved code efficiency and readability. The for loop iteratively processes each loan request using _loanRequests.length directly within the loop's condition. However, accessing the length property of an array in each iteration of a loop can be slightly less efficient than using a locally stored value, especially in larger loops.
Recommendation	Define a local variable to hold the length of the _loanRequests array and use this variable in the for loop condition. By accessing the array length once and storing it in a local variable, the loop avoids repeated property access, potentially reducing some gas costs. A similar loop pattern can also be introduced in the following functions: • investFixedIncomeFundToCompound() • liquidateActiveLoans() • liquidateLeverages()
Status	**Fixed**

Issue	LidoLeverageContract: Unclear error messages in require statements
Line No.	224, 260
Description	During manual review it was found that the require statements in the unwindLeverage() functions could be improved to indicate the specific reason behind a transaction revert.
Recommendation	• The error messages in the above-mentioned lines could be modified to represent the exact status of the position that is required to pass the check. • For instance: “Leverage Contract: Status should be ACTIVE”
Status	**Fixed**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	LeverageLine: Contract includes Hardcoded Addresses
Line No.	18, 21, 25
Description	Keeping in mind the immutable nature of smart contracts, it is not considered a better practice to hardcode any address in the contract before deployment.
Recommendation	Instead of including hardcoded addresses in the contract, initialize those addresses within the constructors at the time of deployment.
Status	**Acknowledged**
Developer's Response	We utilize a cloning method for deploying LeverageLine contracts, making hardcoded addresses necessary. Additionally, the addresses are carefully selected and unlikely to change in the future.

Issue	Avoid use of outdated development tool (Truffle)
Description	The Smart Credit Contract project currently utilizes the Truffle Framework for smart contract development. However, Truffle Framework has become outdated and may not be the optimal choice for current and future smart contract development or testing and security needs.
Recommendation	It is recommended to migrate to some active development tool like Hardhat or Foundry.
Status	**Acknowledged**
Developer's Response	We are already in the process of migrating our codebase from Truffle to Hardhat. However, at the time of your audit, the migration was not yet completed, and therefore, we were unable to implement it with the current baseline provided for auditing.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Intensional Functionality

Issue	SmartCredit.initialize(): Compilation Failure Due to Outdated Role Assignment Method in SmartCredit Contract
Line No.	118-125
Description	*Note: Intentional design to avoid storage collision* The initialize function in the SmartCredit contract, designed to initialize contract state and roles, encounters a compilation error due to the use of an outdated method from the OpenZeppelin library. The contract imports AccessControl from OpenZeppelin but attempts to use _setupRole. This method was available in older versions of OpenZeppelin's AccessControl but is not present in version 5.0.0. This issue leads to a compilation failure, as _setupRole is an undeclared identifier in the context of the current OpenZeppelin version. As a result, the contract cannot be compiled or deployed, rendering it non-functional until the issue is resolved. *Note: While SmartCreditV1 contract is not under the audit scope, the similar issue mentioned above, also exists in the SmartCreditV1 contract's initialize() function.*
Recommendation	The correct method in the latest version is grantRole, which serves a similar purpose but aligns with the updated library. Therefore, replace _setupRole with grantRole in the initialize function to align with the OpenZeppelin 5.0.0 version.
Status	**Acknowledged**
Developer's Response	We acknowledge the known issue of data storage collision encountered upon updating the AccessControl contract to the new version released by OpenZeppelin. This resulted in the usage of an outdated method (_setupRole) in the initialize function, incompatible with the latest OpenZeppelin version 5.0.0.

Automated Testing

```
Total number of contracts in source files: 11
Source lines of code (SLOC) in source files: 621
Number of  assembly lines: 0
Number of optimization issues: 0
Number of informational issues: 87
Number of low issues: 5
Number of medium issues: 0
Number of high issues: 1
ERCs: ERC20

+-----------------+-------------+-------+--------------------+--------------+--------------------+
| Name            | # functions |  ERCS |         ERC20 info | Complex code |           Features |
+-----------------+-------------+-------+--------------------+--------------+--------------------+
| IERC20Permit    |           3 |       |                    |           No |       AbiEncoderV2 |
| IERC20          |           6 | ERC20 |          No Minting |           No |       AbiEncoderV2 |
|                 |             |       | Approve Race Cond. |              |                    |
|                 |             |       |                    |              |                    |
| Address         |           8 |       |                    |           No |           Send ETH |
|                 |             |       |                    |              |       Delegatecall |
|                 |             |       |                    |              |       AbiEncoderV2 |
|                 |             |       |                    |              |           Assembly |
| SafeERC20       |           7 |       |                    |           No |           Send ETH |
|                 |             |       |                    |              | Tokens interaction |
|                 |             |       |                    |              |       AbiEncoderV2 |
| ILiquidator     |           1 |       |                    |           No |       AbiEncoderV2 |
| ECDSA           |           5 |       |                    |           No |          Ecrecover |
|                 |             |       |                    |              |       AbiEncoderV2 |
|                 |             |       |                    |              |           Assembly |
| LoanMath        |           9 |       |                    |           No |       AbiEncoderV2 |
| LeverageStorage |           0 |       |                    |           No |       AbiEncoderV2 |
| ICreditFIFV2    |           1 |       |                    |           No |       AbiEncoderV2 |
| LeverageLine    |          29 |       |                    |           No |        Receive ETH |
|                 |             |       |                    |              |           Send ETH |
|                 |             |       |                    |              | Tokens interaction |
|                 |             |       |                    |              |       AbiEncoderV2 |
|                 |             |       |                    |              |           Assembly |
|                 |             |       |                    |              |       Upgradeable |
+-----------------+-------------+-------+--------------------+--------------+--------------------+
```

```
Total number of contracts in source files: 16
Source lines of code (SLOC) in source files: 897
Number of  assembly lines: 0
Number of optimization issues: 0
Number of informational issues: 98
Number of low issues: 11
Number of medium issues: 4
Number of high issues: 3
ERCs: ERC20

+----------------------+-------------+-------+--------------------+--------------+--------------------+
| Name                 | # functions |  ERCS |          ERC20 info | Complex code |           Features |
+----------------------+-------------+-------+--------------------+--------------+--------------------+
| IERC20Permit         |           3 |       |                    |           No |       AbiEncoderV2 |
| IERC20               |           6 | ERC20 |          No Minting |           No |       AbiEncoderV2 |
|                      |             |       | Approve Race Cond. |              |                    |
|                      |             |       |                    |              |                    |
| Address              |           8 |       |                    |           No |           Send ETH |
|                      |             |       |                    |              |       Delegatecall |
|                      |             |       |                    |              |       AbiEncoderV2 |
|                      |             |       |                    |              |           Assembly |
| SafeERC20            |           7 |       |                    |           No |           Send ETH |
|                      |             |       |                    |              | Tokens interaction |
|                      |             |       |                    |              |       AbiEncoderV2 |
| ISmartCreditRegistry |           8 |       |                    |           No |       AbiEncoderV2 |
| ILiquidator          |           1 |       |                    |           No |       AbiEncoderV2 |
| ECDSA                |           5 |       |                    |           No |          Ecrecover |
|                      |             |       |                    |              |       AbiEncoderV2 |
|                      |             |       |                    |              |           Assembly |
| LoanMath             |           9 |       |                    |           No |       AbiEncoderV2 |
| LeverageStorage      |           0 |       |                    |           No |       AbiEncoderV2 |
| ICreditFIFV2         |           1 |       |                    |           No |       AbiEncoderV2 |
| ICurveFinance        |           1 |       |                    |           No |       AbiEncoderV2 |
| ILido                |           4 |       |                    |           No |        Receive ETH |
|                      |             |       |                    |              |       AbiEncoderV2 |
| ILidoWithdraws       |           5 |       |                    |           No |       AbiEncoderV2 |
| LidoLeverageContract |          45 |       |                    |           No |        Receive ETH |
|                      |             |       |                    |              |           Send ETH |
|                      |             |       |                    |              | Tokens interaction |
|                      |             |       |                    |              |       AbiEncoderV2 |
|                      |             |       |                    |              |           Assembly |
|                      |             |       |                    |              |        Upgradeable |
+----------------------+-------------+-------+--------------------+--------------+--------------------+
```

Concluding Remarks

While conducting the audits of the SmartCredit, it was observed that the system contains High, Medium, and Low severity issues. Our auditors suggest that the developers should resolve issues on priority before moving up with production. The recommendations given will improve the operations of the smart contract.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the SmartCredit platform or its product, nor is this audit investment advice.

Notes:

- Please make sure the code deployed on the mainnet is the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS